

LAMP Stack Installation Guide

Linux • Apache2 • MySQL • PHP • phpMyAdmin — Complete Step-by-Step Setup

Overview

This document walks through installing and configuring a full **LAMP** stack (Linux, Apache, MySQL, PHP) plus **phpMyAdmin** on a Debian/Ubuntu-based Linux server. Commands are written for **Ubuntu Server 22.04/24.04 LTS**, but work with minor changes on any Debian-based distribution.

Prerequisite: A Linux server/VM with `sudo` access and internet connectivity, and a non-root user.

Step 1 — Update the System

Always start with a fully updated package index and upgraded packages.

```
sudo apt update && sudo apt upgrade -y
```

Command	Purpose
<code>apt update</code>	Refreshes the local package index from repositories
<code>apt upgrade -y</code>	Installs available upgrades without prompting

Step 2 — Install Apache2 (Web Server)

```
sudo apt install apache2 -y
```

Enable Apache to start on boot and start it now:

```
sudo systemctl enable apache2
sudo systemctl start apache2
```

Check status:

```
sudo systemctl status apache2
```

Allow Apache through the firewall (if UFW is active):

```
sudo ufw allow in "Apache Full"
sudo ufw status
```

Verify installation — open a browser and go to:

```
http://your-server-ip/
```

You should see the default **Apache2 Ubuntu Default Page**.

Step 3 — Install MySQL (Database Server)

```
sudo apt install mysql-server -y
```

Enable and start the service:

```
sudo systemctl enable mysql
sudo systemctl start mysql
```

Secure the installation (sets root password, removes anonymous users, disables remote root login, removes test database):

```
sudo mysql_secure_installation
```

You will be prompted for: 1. Whether to configure the *Validate Password* plugin 2. Set a root password 3. Remove anonymous users → **Yes** 4. Disallow root login remotely → **Yes** 5. Remove test database → **Yes** 6. Reload privilege tables → **Yes**

Create a dedicated admin user (recommended over using root everywhere):

```
sudo mysql
CREATE USER 'lampadmin'@'localhost' IDENTIFIED BY 'StrongPassword123!';
GRANT ALL PRIVILEGES ON *.* TO 'lampadmin'@'localhost' WITH GRANT OPTION;
FLUSH PRIVILEGES;
EXIT;
```

Verify installation:

```
mysql -u lampadmin -p -e "SELECT VERSION();" 
```

Step 4 — Install PHP

Install PHP with the modules Apache and MySQL need:

```
sudo apt install php libapache2-mod-php php-mysql php-cli php-curl php-gd php-mbstring php-xml php-zip -y
```

Restart Apache to load the PHP module:

```
sudo systemctl restart apache2
```

Verify PHP version:

```
php -v
```

Create a test file to confirm Apache is serving PHP correctly:

```
sudo nano /var/www/html/info.php
```

Add:

```
<?php
phpinfo()
?>
```

Visit <http://your-server-ip/info.php> — you should see the PHP configuration page.

Security note: Delete `info.php` after testing, since it exposes server configuration details.

```
sudo rm /var/www/html/info.php
```

Step 5 — Install phpMyAdmin

```
sudo apt install phpmyadmin -y
```

During the installer prompts:

Prompt	Recommended Choice
Web server to configure	Select apache2 (press Space to select, then Tab + Enter)
Configure database for phpMyAdmin with dbconfig-common	Yes
MySQL application password for phpMyAdmin	Set a strong password

Enable required PHP extension and Apache config (if not auto-enabled):

```
sudo phpenmod mbstring
sudo a2enconf phpmyadmin
sudo systemctl restart apache2
```

If phpMyAdmin was not linked automatically, create a symlink manually:

```
sudo ln -s /etc/phpmyadmin/apache.conf /etc/apache2/conf-available/phpmyadmin.conf
sudo a2enconf phpmyadmin
sudo systemctl reload apache2
```

Access phpMyAdmin:

```
http://your-server-ip/phpmyadmin
```

Log in using your MySQL user (e.g., `lampadmin`) and its password.

Step 6 — Configure a Virtual Host (Optional but Recommended)

Instead of using `/var/www/html` directly, set up a dedicated site.

```
sudo mkdir -p /var/www/mysite.local/public_html
sudo chown -R $USER:$GROUP /var/www/mysite.local/public_html
sudo chmod -R 755 /var/www/mysite.local
```

Create a config file:

```
sudo nano /etc/apache2/sites-available/mysite.local.conf
```

```
<VirtualHost *:80>
    ServerAdmin admin@mysite.local
    ServerName mysite.local
    ServerAlias www.mysite.local
    DocumentRoot /var/www/mysite.local/public_html
    ErrorLog ${APACHE_LOG_DIR}/mysite.local_error.log
    CustomLog ${APACHE_LOG_DIR}/mysite.local_access.log combined
</VirtualHost>
```

Enable the site and disable the default (optional):

```
sudo a2ensite mysite.local.conf
sudo a2dissite 000-default.conf
sudo apache2ctl configtest
sudo systemctl reload apache2
```

Add a local hosts entry for testing (on your local machine, not the server):

```
your-server-ip mysite.local
```

Step 7 — Enable HTTPS with Let's Encrypt (Optional, Production Recommended)

```
sudo apt install certbot python3-certbot-apache -y
sudo certbot --apache -d yourdomain.com -d www.yourdomain.com
```

Certbot will automatically configure SSL and set up auto-renewal via a systemd timer:

```
sudo systemctl status certbot.timer
```

Step 8 — Install WordPress

WordPress needs its own database, a dedicated MySQL user, the WordPress files, and some Apache/PHP tuning.

8.1 Create the WordPress Database and User

```
sudo mysql -u lampadmin -p
```

```
CREATE DATABASE wordpress_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'wp_user'@'localhost' IDENTIFIED BY 'StrongWpPassword123!';
GRANT ALL PRIVILEGES ON wordpress_db.* TO 'wp_user'@'localhost';
FLUSH PRIVILEGES;
EXIT;
```

8.2 Install Required PHP Extensions for WordPress

```
sudo apt install php-curl php-xml php-mbstring php-zip php-imagick php-soap php-intl
php-gd -y
sudo systemctl restart apache2
```

8.3 Download and Extract WordPress

```
cd /tmp
curl -O https://wordpress.org/latest.tar.gz
tar -xvzf latest.tar.gz
```

Move it into your site's document root (using the virtual host created in Step 6, adjust path as needed):

```
sudo rsync -av /tmp/wordpress/ /var/www/mysite.local/public_html/
```

(If installing straight into the default site instead, use `/var/www/html/` as the destination.)

8.4 Set Correct Ownership and Permissions

```
sudo chown -R www-data:www-data /var/www/mysite.local/public_html
sudo find /var/www/mysite.local/public_html -type d -exec chmod 755 {} \;
sudo find /var/www/mysite.local/public_html -type f -exec chmod 644 {} \;
```

8.5 Configure `wp-config.php`

```
cd /var/www/mysite.local/public_html
sudo cp wp-config-sample.php wp-config.php
sudo nano wp-config.php
```

Update the database section:

```
define( 'DB_NAME', 'wordpress_db' );
define( 'DB_USER', 'wp_user' );
define( 'DB_PASSWORD', 'StrongWpPassword123!' );
define( 'DB_HOST', 'localhost' );
```

Add unique security keys — generate them at <https://api.wordpress.org/secret-key/1.1/salt/> and paste them in place of the placeholder block in `wp-config.php`.

Enable direct filesystem method (for plugin/theme installs from the dashboard):

```
define( 'FS_METHOD', 'direct' );
```

8.6 Enable `mod_rewrite` for Pretty Permalinks

```
sudo a2enmod rewrite
```

Ensure `AllowOverride All` is set for the site's `<Directory>` block (done already if you followed Step 6), then:

```
sudo systemctl restart apache2
```

8.7 Run the Web Installer

Visit:

```
http://your-server-ip/
```

or

```
http://mysite.local/
```

Follow the on-screen setup wizard: choose site title, admin username, admin password, and admin email, then click **Install WordPress**.

8.8 Post-Install Hardening (Recommended)

- In **Settings → Permalinks**, select “Post name” for clean URLs (this writes a `.htaccess` file automatically).
- Delete the default `readme.html`, `license.txt` from the WordPress root if desired.
- Install an SSL certificate (Step 7) and force HTTPS via **Settings → General**.
- Set up automatic backups (database dump + files) before installing plugins/themes.
- Disable file editing from the dashboard:

```
define( 'DISALLOW_FILE_EDIT', true );
```

Step 9 — Final Verification Checklist

Component	Check Command	Expected Result
Apache	<code>sudo systemctl status apache2</code>	<code>active (running)</code>
MySQL	<code>sudo systemctl status mysql</code>	<code>active (running)</code>
PHP	<code>php -v</code>	Shows installed PHP version
phpMyAdmin	Visit <code>/phpmyadmin</code> in browser	Login page loads
WordPress	Visit site URL in browser	WordPress site or setup wizard loads
Firewall	<code>sudo ufw status</code>	Apache Full is allowed

Your LAMP stack with phpMyAdmin and WordPress is now fully installed and ready for use.